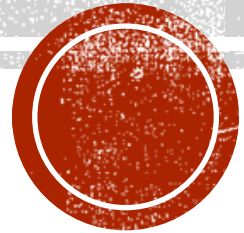
LECTURE 05

Creating Interactive applications



WHAT ARE INTERACTIVE APPLICATIONS?

- Interactive applications allows the user to interact with the application at runtime.
- We implement interactive applications in Flutter using the Stateful class widgets.
- In this week we will learn how to take user input using the TextField widget.
- We will also learn how to distribute are classes across multiple libraries(files).

THE STATEFUL WIDGET CLASS

- Since we will be creating an interactive application, we need to use Stateful widgets.
- A Stateful widget has a Stateful class and a state class.
- The state class will redraw the widget when its state changes.
- We inform flutter that the state of the widget has changed by calling the **setState** method.

THE STATEFUL WIDGET CLASS

The StatefulWidget class has the below structure:

```
class SampleStaefulWidget extends StatefulWidget {
  const SampleStaefulWidget({Key? key}) : super(key: key);

  @override
  State<SampleStaefulWidget> createState() => _SampleStaefulWidgetState();
}
// the State class redraws the widget when we call the setState method
class _SampleStaefulWidgetState extends State<SampleStaefulWidget> {
  @override
  Widget build(BuildContext context) {
    return Container();
  }
}
```

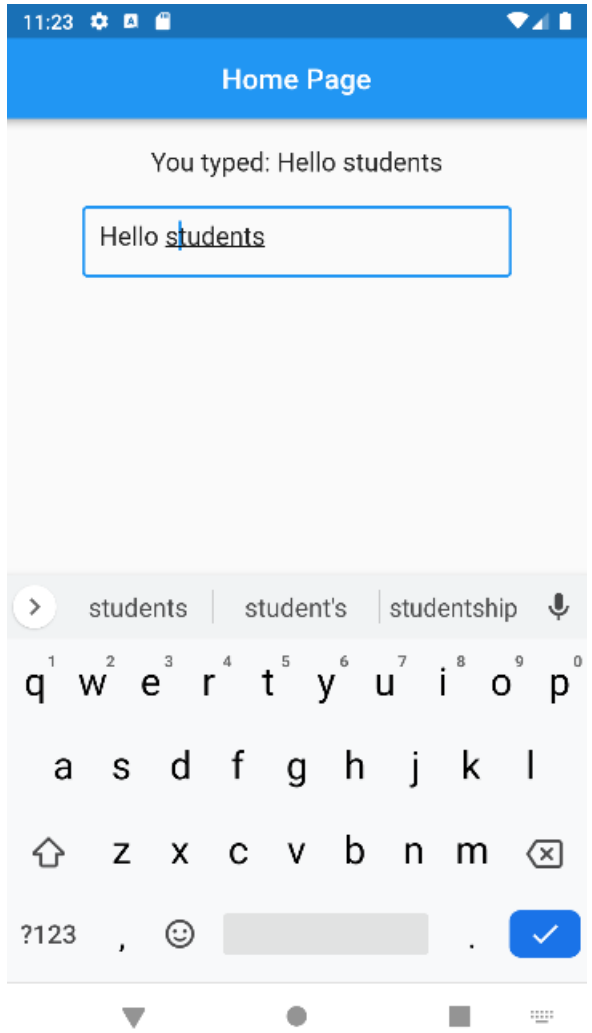
THE TEXTFIELD WIDGET

The TextField widget allows an application to take input from the user. It includes many fields, below are some of them:

- style: defines the text style using a TextStyle widget.
- decorations: Define the TextField style, such as border and hintText, using the InputDecoration widget.
- onChanged: defines an anonymous method that takes a String argument and is called when the TextField’s text is changed.
- onSubmitted: defines an anonymous method that takes a String argument and is called when the user finishes editing text.

OUR FIRST APPLICATION

- The first application we will be creating, will include a Text and TextField widget
- The text the user will enter in the TextField will be displayed in the Text widget.



CREATING THE APPLICATION

Using android studio, create a new flutter application. In the main.dart file, add the below code:

```
import 'package:flutter/material.dart';
import 'home.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      title: 'CSCI410 Week 4 Project',
      debugShowCheckedModeBanner: false,
      home: Home()
    );
  }
}
```

- We will be using a Stateful widget called Home.
- The Home widget is define in the home.dart file, we will create later.
- That's why why imported the home.dart library.

CREATING HOME.DART FILE

Create a new file called “home.dart”. Place the below code inside of it. You can use the st command to create a StatefulWidget called home. Don’t forget to import the material.dart package.

```
import 'package:flutter/material.dart';

class Home extends StatefulWidget {
  const Home({Key? key}) : super(key: key);

  @override
  State<Home> createState() => _HomeState();
}

class _HomeState extends State<Home> {
  @override
  Widget build(BuildContext context) {
    return Container();
  }
}
```

CREATING HOME.DART FILE

Add the below code as the return widget of the _HomeState build method.

```
@override
  Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(
          title: const Text('Home Page'),
          centerTitle: true,
        ),
        body: Container()
    );
  }
```

ADDING THE TEXT AND TEXTFIELD WIDGETS

Replace the Container in for the Body field with the below code. The onChanged method calls the **updateText** function. We will add this function later.

```
Center(
          child: Column(
            children: [
              const SizedBox(height: 20.0),
              Text('You typed: $_text', style: const TextStyle(fontSize: 18.0)),
              const SizedBox(height: 20.0,),
              SizedBox(width: 300.0, height: 50.0,
                child: TextField(
                  style: const TextStyle(fontSize: 18.0),
                  decoration: const InputDecoration(
                      border: OutlineInputBorder(), hintText: 'Enter some text'
                  ),
                  onChanged: (text) { updateText(text);}, // call the updateText function
                ),
              ),
            ],
          ),
        )
```

CODE EXPLAINED

- We added the TextField inside a SizedBox widget to give it a width and height
- We gave the TextField a border using the InputDecoration widget.
- We gave it a font size using the TextStyle widget.
- The onChanged method is called every time the text in the TextField is changed. It calls an updateText method to update the text inside the Text widget. We will create the updateText method in the next slide.

Adding the updateText method and text field

Modify the _HomeState class to add the below code:

```
class _HomeState extends State<Home> {
  // A field to hold the TextField's text
  String _text = '';
  // The updateText method will be called when the text in the TextField changes
  void updateText(String text) {
    // we need to call the setState method, so that the state class will redraw the widget
    setState(() {
      _text = text;
    });
  }
```

CREATING A CUSTOM TEXTFIELD WIDGET

We will create a custom widget for the TextField. This way we can use the design multiple times in our project.

```
class MyTextField extends StatelessWidget {
  Function(String) f; // hold a variable function
  String hint; // holds the hintText of the TextField

  MyTextField({required this.hint, required this.f, super.key,});

  @override
  Widget build(BuildContext context) {
    return SizedBox(width: 300.0, height: 50.0,
      child: TextField(
        style: const TextStyle(fontSize: 18.0),
        decoration: InputDecoration(
            border: const OutlineInputBorder(), hintText: hint
        ),
        onChanged: (text) { f(text);}, // call the variable function
      ),
    );
  }
}
```

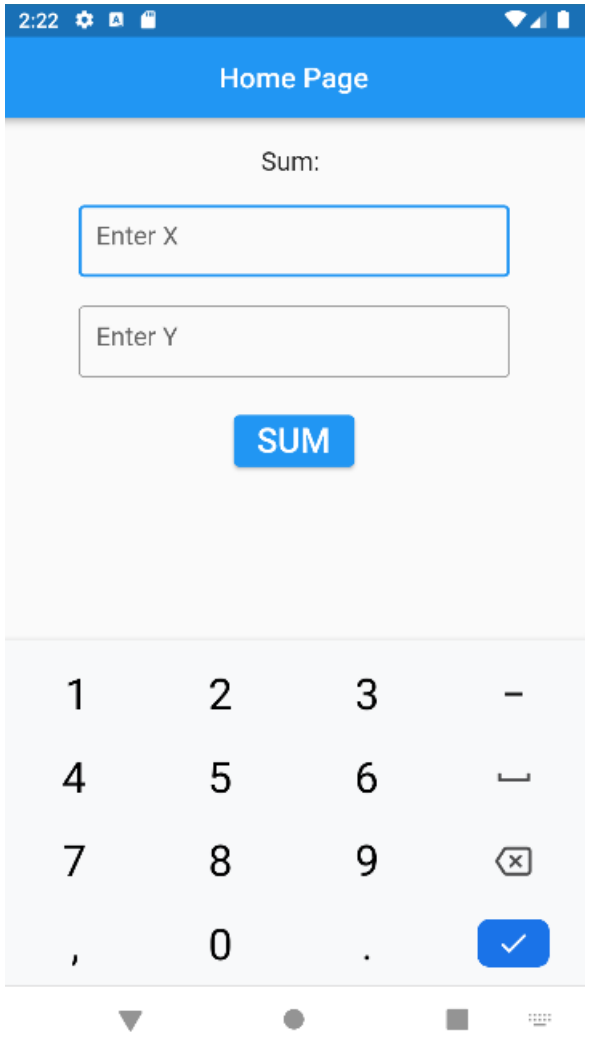
CREATING A CUSTOM TEXTFIELD WIDGET

Now we can call our custom class as shown below, passing to it the updateText function. This is called a **callback function**. We also pass to it the hint text.

```
body: Center(
          child: Column(
            children: [
              const SizedBox(height: 20.0),
              Text(‘You typed: $_text', style: const TextStyle(fontSize: 18.0)),
              const SizedBox(height: 20.0),
              MyTextField(f: updateText, hint: 'Enter some text'),
            ],
          ),
        )
```

PART 2 APPLICATION

Now we will create an application that includes 2 TextFields and a Button. The user will enter X and Y and then the application will display their sum in a Text widget.



MODIFIED _HOMESTATE CLASS

Now we will modify the state class to add the below code:

```
class _HomeState extends State<Home> {
  String _text = '';
  double _x = -1, _y = -1;

  void updateText() {
    setState(() {
      if (_x == -1 || _y == -1) {
        _text = 'Please fill all fields';
      }
      else {
        _text = (_x + _y).toString();
      }
    });
  }
```

```
void updateX(String x) {
  if (x.trim() == '') {
    _x = -1;
  }
  else {
    _x = double.parse(x);
  }
}

void updateY(String y) {
  if (y.trim() == '') {
    _y = -1;
  }
  else {
    _y = double.parse(y);
  }
}
```

CHANGING THE DESIGN

Now we will modify the body of the Scaffold as shown below:

```
body: Center(
          child: Column(
            children: [
              const SizedBox(height: 20.0),
              Text('Sum: $_text', style: const TextStyle(fontSize: 18.0)),
              const SizedBox(height: 20.0),
              MyTextField(f: updateX, hint: 'Enter X'),
              const SizedBox(height: 20.0),
              MyTextField(f: updateY, hint: 'Enter Y'),
              const SizedBox(height: 20.0),
              ElevatedButton(onPressed: () {updateText();}, child: Text('SUM', style: TextStyle(fontSize:
24.0),))
            ],
          ),
        )
```

THE ELEVATEDBUTTON WIDGET

The ElevetaedButon widget draws a button with a shade. It creates an increased shade effect when you press on it. It has many fields, below are some of them.

- child: usually it is a Text widget.
- onPressed: takes an anonymous function that is called when the button is pressed.
- You can change the design of the button to be round for example. You can try this as an exercise by yourself.